# L02 - Transformer 全景图

> **"注意力就是一切"**

---

## 📌 本节目标

1. 理解为什么 Transformer 取代了 RNN，成为 NLP 的基石
2. 掌握 Transformer 架构的整体结构（Embedding → Attention → FFN → Output）
3. 用直觉理解 Self-Attention 和多头注意力的工作方式
4. 理解 Attention 公式中每一步的含义
5. 掌握残差连接、Layer Normalization、Positional Encoding 的作用

---

## 📚 前置知识

- 阅读完 L01，理解"下一个词预测"和 Decoder-Only 架构的基本概念
- 知道矩阵乘法是两个表格的运算（不需要会算，知道存在即可）
- 了解"概率"和"向量"的直觉含义

---

## 正文讲解

### 1. 为什么需要 Transformer? ——RNN 的困境

在 Transformer 出现之前（2017 年以前），处理语言的主流模型是 **RNN（循环神经网络）**。

**RNN 是怎么工作的?**

RNN 像一个**流水线工人**，一次只处理一个词，每处理完一个词就把"记忆"传给下一步：

```
"我" → [处理] → 记忆1
"喜" → [处理 + 记忆1] → 记忆2
"欢" → [处理 + 记忆2] → 记忆3
"编" → [处理 + 记忆3] → 记忆4
"程" → [处理 + 记忆4] → 最终输出
```

**RNN 的两大致命问题**

**问题 1：长距离依赖（记忆衰退）**

就像"传话游戏"——消息经过太多人之后就走样了。

```
"小明今天早上吃了早饭，然后去了学校，上了数学课，
 接着上了英语课，中午吃了食堂的红烧肉，下午……
 他昨天说的那件事是关于___"
```

当要填最后的空时，RNN 已经"忘记"了小明昨天说了什么——因为中间经过了太多步，记忆被不断稀释。

**问题 2：无法并行计算（速度瓶颈）**

RNN 必须**按顺序**处理——第二个词必须等第一个词处理完才能开始。一篇 1000 词的文章，需要 1000 步串行计算。

> **类比**：RNN 就像排队过一座**单车道的桥**，每次只能过一辆车，后面的车再着急也得等着。

**Transformer 的答案**

2017 年，Google 团队发表了划时代的论文《Attention Is All You Need》，提出了 **Transformer** 架构。它同时解决了 RNN 的两大问题：

- **长距离依赖** → 用 Attention 机制，每个词可以直接"看到"句子中的所有其他词
- **无法并行** → 不再逐词处理，所有词同时计算

> **类比**：Transformer 就像一座**立交桥**，所有方向的车辆可以同时通行，互不干扰。

## 2. Transformer 架构全景图——餐厅比喻

我们用一家**高档餐厅**来类比 Transformer 的工作流程：

```
┌──────────────────────────────────────┐
│              Transformer 餐厅          │
│                                      │
│  📋 Embedding (菜单翻译)              │
│    顾客点菜 → 翻译成厨房能理解的语言      │
│                                      │
```

```
│   📍 Positional Encoding (座位号)          │
│     给每道菜贴上"第几道"的标签              │
│                                            │
│   🔄 × N 层 (N 个厨师团队轮流加工)         │
│   ┌──────────────────────────────┐         │
│   │   👨‍🍳 Self-Attention (厨师间沟通)  │      │
│   │     每个厨师都去看看其他厨师在做什么  │      │
│   │     → 决定自己该加什么调料          │      │
│   │                                    │      │
│   │   🍳 Feed-Forward (各展手艺)       │      │
│   │     每个厨师用自己的独门技法加工食材  │      │
│   │                                    │      │
│   │   ➕ 残差连接 (保留原味)           │      │
│   │     加工后与原始食材混合，不丢原味   │      │
│   │                                    │      │
│   │   📏 Layer Norm (质量检查)         │      │
│   │     确保每道菜的味道在标准范围内     │      │
│   └──────────────────────────────┘         │
│                                            │
│   🍽️ Output (上菜)                         │
│     把最终结果翻译回顾客能懂的语言          │
└────────────────────────────────────────────┘
```

## 3. Encoder 和 Decoder 的区别

原始 Transformer 有两个部分：

| 组件 | 功能 | 类比 |
|---|---|---|
| **Encoder** | 读懂输入 | 听你说话的"理解者" |
| **Decoder** | 生成输出 | 组织语言的"表达者" |

**关键区别**：

- **Encoder**：每个词可以看到**所有**其他词（双向的）
- **Decoder**：每个词只能看到**它前面的**词（单向的，因果遮罩）

```
Encoder (双向):
  "我 喜欢 编程" → "喜欢"可以同时看到"我"和"编程"

Decoder (单向/因果):
  "我 喜欢 编程" → "喜欢"只能看到"我"，看不到"编程"
                 → "编程"可以看到"我"和"喜欢"
```

### 为什么 Decoder 要"遮住"后面的词?

因为在生成时，后面的词还没有被生成出来！模型要预测"下一个词"，所以在训练时也必须模拟这种"只能看前面"的场景。

MiniMind 使用的是 **纯 Decoder 架构**（Decoder-Only），没有 Encoder 部分。所有输入和输出都在同一个序列中处理。

## 4. Self-Attention：每个词都在"看"其他词

Self-Attention 是 Transformer 最核心的创新。

**直觉理解**

想象你在读这句话：

> "小猫坐在垫子上，**它**很舒服"

当你读到"它"时，你的大脑会**回头看**整个句子，快速判断出"它"指的是"小猫"而不是"垫子"。你给"小猫"分配了很高的**注意力权重**，给"垫子"分配了较低的权重。

Self-Attention 做的就是同样的事情——对于句子中的每个词，计算它应该"关注"其他词的程度。

```
输入："小猫 坐在 垫子 上 它 很 舒服"

当处理"它"时的注意力权重：
  "小猫" → 0.65 ⭐ （最高关注）
  "坐在" → 0.05
  "垫子" → 0.12
  "上"   → 0.03
  "它"   → 0.08
  "很"   → 0.04
  "舒服" → 0.03
```

### Q、K、V 三剑客

Self-Attention 有三个核心角色：**Query（Q）、Key（K）、Value（V）**

用"查百科全书"来类比：

| 角色 | 类比 | 含义 |
| --- | --- | --- |
| **Query（查询）** | 你的问题 | "我想找什么？" |
| **Key（键）** | 百科全书每个词条的标题 | "我这里有什么？" |
| **Value（值）** | 百科全书每个词条的内容 | "如果你来找我，我给你这些信息" |

工作流程：

```
第 1 步：每个词生成自己的 Q、K、V
  "小猫" → Q_小猫，K_小猫，V_小猫
  "坐在" → Q_坐在，K_坐在，V_坐在
  ...

第 2 步：用 Q 去和所有的 K 做匹配（点积）
  Q_它 · K_小猫 = 高分 ⭐ (很匹配！)
  Q_它 · K_垫子 = 低分
  Q_它 · K_坐在 = 低分

第 3 步：把匹配分数转化为注意力权重（softmax）
  [高分，低分，低分，...] → [0.65, 0.12, 0.05, ...]

第 4 步：用权重加权求和所有的 V
  输出_它 = 0.65 × V_小猫 + 0.12 × V_垫子 + 0.05 × V_坐在 + ...
```

这样，"它"这个词的表示就融入了它所关注的其他词（主要是"小猫"）的信息。

## 5. Attention 公式详解

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

别被公式吓到！我们逐步分解：

### 第 1 步：$QK^T$ （计算匹配度）

Q 和 K 做矩阵乘法，得到一个"注意力分数矩阵"。

> **直觉**：Q 是"我在找什么"，K 是"我有什么"。两者越相似，点积（内积）分数越高。就像两个人的兴趣越接近，聊天就越投缘。

```
假设句子有 4 个词，每个词的向量维度是 3：
Q = [[1,0,1], [0,1,0], [1,1,0], [0,0,1]]  (4×3)
K = [[1,0,1], [0,1,0], [1,1,0], [0,0,1]]  (4×3)

QK^T = 4×4 的矩阵，每个位置 (i,j) 表示：第 i 个词对第 j 个词的关注度
```

### 第 2 步：除以 $\sqrt{d_k}$ （缩放）

$d_k$

是 Key 向量的维度。为什么要除以它的平方根?

> **直觉**：如果向量维度很大，点积的结果数值也会很大。数值太大的话，softmax 会变得"太极端"——一个分数接近 1，其他全接近 0。除以 $\sqrt{d_k}$ 就像是把温度调到合适范围，让注意力分布不至于"只盯着一个人看"。

```
假设 d_k = 64
原始分数: [128, 5, 3, 2]
除以 √64=8: [16, 0.625, 0.375, 0.25]
softmax 变得更平滑，不会过度集中
```

**面试常考点**：不除以 $\sqrt{d_k}$，softmax 的输入值会过大，梯度会非常小（softmax 在极端值处梯度趋近于 0），导致训练不稳定。

**第 3 步：**softmax**（转化为概率）**

将分数归一化为概率分布（所有值在 0-1 之间，总和为 1）。

```
缩放后的分数: [16, 0.625, 0.375, 0.25]
softmax 后:   [0.99, 0.003, 0.002, 0.005]
```

**第 4 步：乘以 $V$（加权聚合）**

用注意力权重对 V 做加权求和，得到最终输出。

```
输出 = 0.99 × V_1 + 0.003 × V_2 + 0.002 × V_3 + 0.005 × V_4
```

每个词的输出都是所有词的 Value 的加权混合，权重由 Q 和 K 的匹配程度决定。

## 6. 多头注意力：多个角度看同一句话

**Multi-Head Attention（多头注意力）** 是 Transformer 的另一个关键设计。

**为什么需要多个"头"?**

一个注意力头只能从一个角度理解句子。就像看一幅画：

- 头 1 关注**语法关系**："它"→"小猫"（代词指代）
- 头 2 关注**位置关系**：相邻的词之间的联系

- 头 3 关注**语义关系**："舒服"→"垫子"（垫子让人舒服）

> **类比**：单头注意力像是只用一只眼睛看世界，多头注意力像是用多只眼睛从不同角度同时观察，然后把所有角度的信息综合起来。

**具体实现**

```
原始向量维度 d_model = 768
注意力头数 n_heads = 8
每个头的维度 head_dim = 768 / 8 = 96

步骤:
1. 把 768 维的 Q/K/V 各切成 8 份，每份 96 维
2. 每个头独立计算 Attention
3. 把 8 个头的输出拼接回 768 维
4. 通过一个线性层（投影矩阵）做最终变换
```

MiniMind 使用了 **GQA（Grouped-Query Attention，分组查询注意力）**：

- Q 有 8 个头（`num_attention_heads=8`）
- K 和 V 只有 4 个头（`num_key_value_heads=4`）
- 每 2 个 Q 头共享 1 个 K/V 头

这样可以减少 K/V 的参数量和显存占用，同时基本不损失性能——这是 LLaMA 等现代模型广泛采用的技术。

## 7. 残差连接 (Residual Connection)

**是什么?**

残差连接就是**把层的输入直接加到层的输出上**：

output = Layer($x$) + $x$

**为什么需要?**

> **类比**：就像做菜时，即使你加了新的调料（Layer 的变换），也要保留食材的原味（x）。这样，即使调料放多了（某一层学偏了），原味还在，不至于完全变味。

技术上的好处：

1. **解决梯度消失**：在反向传播时，梯度可以直接通过"捷径"流回去，不会在深层网络中消失
2. **让深层网络可训练**：没有残差连接的话，超过 20 层的网络几乎训练不动
3. **保底不退步**：最差情况下，如果某一层没学到有用的东西，至少输出等于输入（Layer($x$)≈0 时）

## 8. Layer Normalization（层归一化）

### 是什么?

Layer Norm 对每个样本的特征做归一化，让它们的均值为 0、方差为 1。

### 为什么需要?

> **类比**：就像考试后进行"标准化评分"。不同题目难度不同，直接比较原始分不公平。归一化后，所有维度的数值都在类似的范围内，模型训练更稳定。

MiniMind 采用的是 **RMSNorm**（Root Mean Square Normalization），这是 Layer Norm 的简化版本，去掉了减去均值的步骤，只做缩放：

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}} \times \gamma$$

RMSNorm 比标准 LayerNorm 计算更快，效果相当，被 LLaMA、MiniMind 等模型广泛使用。

MiniMind 中的位置是 **Pre-Norm**（先归一化再做 Attention/FFN），而不是原始 Transformer 的 Post-Norm。

## 9. Positional Encoding（位置编码）

### 为什么需要?

Attention 机制有一个"缺陷"——它是**无序的**。

```
"小猫追小狗" 和 "小狗追小猫"
对于纯 Attention 来说，这两句话的关系是一样的！
```

因为 Attention 只看"谁和谁相关"，不看"谁在前谁在后"。所以我们需要给每个词加上**位置信息**。

### MiniMind 使用的 RoPE（旋转位置编码）

MiniMind 不是使用原始 Transformer 论文中的正弦位置编码，而是采用了更先进的 **RoPE（Rotary Position Embedding）**。

> **直觉**：RoPE 的核心思想是"旋转"——把位置信息编码为向量空间中的旋转角度。两个位置之间的距离越远，它们之间的旋转角度就越大。这样做的好处是：(1) 位置信息自然融入了 Q 和 K 的点积运算中；(2) 可以灵活外推到训练时没见过的更长序列。

RoPE 的效果：

```
位置 0 的词向量 → 旋转 0°
位置 1 的词向量 → 旋转 θ°
位置 2 的词向量 → 旋转 2θ°
...
```

两个词的注意力分数只取决于它们的**相对位置**（旋转角度差），而不是绝对位置。

## 10. Feed-Forward Network (FFN)

在 Attention 之后，每个 token 会经过一个 **前馈神经网络 (FFN)**。

**作用**

如果说 Attention 是"收集信息"（每个词去看其他词），FFN 就是"消化信息"（用自己的"专业知识"处理收集到的信息）。

> **类比**：Attention 像是开会讨论，每个人听了别人的意见；FFN 像是会后各自回工位深入思考和处理。

**MiniMind 使用的 SwiGLU**

MiniMind 的 FFN 不是简单的两层全连接，而是使用了 **SwiGLU** 激活函数：

$$\text{FFN}(x) = W_{\text{down}} \cdot (\text{SiLU}(W_{\text{gate}} \cdot x) \odot W_{\text{up}} \cdot x)$$

其中 $\odot$ 表示逐元素乘法。SwiGLU 通过一个"门控"机制让模型学会选择性地传递信息，效果优于传统的 ReLU。

对应 MiniMind 代码（`model/model_minimind.py`）中的 `FeedForward` 类：

```
class FeedForward(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.gate_proj = nn.Linear(config.hidden_size, config.intermediate_size,
bias=False)
        self.down_proj = nn.Linear(config.intermediate_size, config.hidden_size,
bias=False)
        self.up_proj = nn.Linear(config.hidden_size, config.intermediate_size, bias=False)

    def forward(self, x):
        return self.down_proj(F.silu(self.gate_proj(x)) * self.up_proj(x))
```

## 11. 把它们组合起来——MiniMind 的一个 Block

MiniMind 的每个 Transformer Block（`MiniMindBlock`）结构如下：

```
输入 x
  │
  ├── RMSNorm(x) → Self-Attention → + x    (残差连接)
  │
  ├── RMSNorm(x) → FFN (SwiGLU)   → + x    (残差连接)
  │
  输出
```

这就是 **Pre-Norm + 残差连接** 的标准模式。MiniMind 有 8 层这样的 Block 堆叠在一起（`num_hidden_layers=8`）。

## 12. MiniMind 的完整前向流程

```
输入文字: "今天天气"
    │
    ▼
[Tokenizer] → token_ids: [234, 567, 567, 891]
    │
    ▼
[Embedding] → 每个 token 变成 768 维向量
    │
    ▼
[RoPE 位置编码] → 给每个向量加上位置信息
    │
    ▼
[Block 1] → RMSNorm → Attention(GQA) → 残差 → RMSNorm → FFN(SwiGLU) → 残差
[Block 2] → 同上
[Block 3] → 同上
  ...
[Block 8] → 同上
    │
    ▼
[RMSNorm] → 最后做一次归一化
```

```
    │
    ▼
[lm_head] → 768 维 → 6400 维（vocab_size，每个 token 一个分数）
    │
    ▼
[softmax + 采样] → 选出下一个 token
```

---

## 📂 对应 MiniMind 源码

| 概念 | 对应文件 | 关键代码 |
|---|---|---|
| 注意力机制（GQA + RoPE） | `model/model_minimind.py` | `class Attention` — Q/K/V 投影、RoPE 旋转、因果 mask、GQA repeat_kv |
| 前馈网络（SwiGLU） | `model/model_minimind.py` | `class FeedForward` — gate_proj / up_proj / down_proj + SiLU 激活 |
| Transformer Block | `model/model_minimind.py` | `class MiniMindBlock` — Pre-RMSNorm + Attention + FFN + 残差连接 |
| 完整模型 | `model/model_minimind.py` | `class MiniMindForCausalLM` — Embedding + N × Block + lm_head |
| RMSNorm | `model/model_minimind.py` | `class RMSNorm` — 只做缩放的简化版 LayerNorm |
| RoPE 位置编码 | `model/model_minimind.py` | `precompute_freqs_cis()` 和 `apply_rotary_emb()` 函数 |

---

## 🎤 面试考点

### Q1: Attention 公式中为什么要除以 √d_k?

**答**：当 Key 的维度 $d_k$ 较大时，$QK^T$ 的点积结果会随维度增大而增大（方差约为 $d_k$）。如果不缩放，softmax 的输入值会非常大，导致梯度趋近于 0（梯度消失），模型无法有效学习。除以 $\sqrt{d_k}$ 将方差控制在 1 附近，使 softmax 的输出不会过于集中在某一个位置，保证梯度正常流动。这种做法也被称为 Scaled Dot-Product Attention。

### Q2: 残差连接的作用是什么？如果去掉会怎样?

**答**：残差连接将层的输入直接加到层的输出上（$y = F(x) + x$），有三个主要作用： 1. **缓解梯度消失**：梯度可以通过"短路"直接传回浅层，避免在深层网络中消失； 2. **使深层网络可训练**：

ResNet 论文证明，没有残差连接时，超过一定深度的网络性能反而下降（退化问题）； 3. **保证下限**：即使某一层完全没学到有用的变换，输出至少等于输入，不会变差。 如果去掉残差连接，Transformer 在 8 层以上时几乎无法训练，损失函数会不收敛。

### Q3: Transformer 相比 RNN 的并行性优势体现在哪里?

**答**：RNN 处理序列是串行的——必须先处理第 1 个 token，拿到隐藏状态后才能处理第 2 个，依此类推。序列长度为 N，则需要 N 步串行计算。Transformer 的 Self-Attention 一次性计算所有 token 之间的关系（$QK^T$ 是一次矩阵乘法），序列中的所有位置可以并行处理。时间复杂度从 $O(N)$ 步串行降为 $O(1)$ 步并行（但空间复杂度是 $O(N^2)$，因为要存储 $N \times N$ 的注意力矩阵）。这使得 Transformer 在 GPU 上的训练速度远快于 RNN。

### Q4: 什么是 GQA（Grouped-Query Attention）？MiniMind 为什么用它?

**答**：标准多头注意力（MHA）中，Q、K、V 各有 $h$ 个头。GQA 让 K 和 V 的头数少于 Q 的头数——多个 Q 头共享同一组 K/V 头。MiniMind 中 Q 有 8 个头，K/V 只有 4 个头，每 2 个 Q 头共享 1 组 K/V。好处是：(1) 减少 K/V 的参数量和 KV Cache 的显存占用（推理时尤为重要）；(2) 性能损失极小，因为 K/V 的多样性需求低于 Q。GQA 是 MHA（每组 1 个 Q 头）和 MQA（所有 Q 头共享 1 组 K/V）的折中方案。

### Q5: Pre-Norm 和 Post-Norm 有什么区别？MiniMind 用哪种?

**答**：Post-Norm（原始 Transformer）在子层之后做 LayerNorm：$y = \text{Norm}(F(x) + x)$。Pre-Norm 在子层之前做：$y = F(\text{Norm}(x)) + x$。MiniMind 采用 Pre-Norm（使用 RMSNorm）。Pre-Norm 的优势：(1) 训练更稳定，不需要 warm-up；(2) 梯度流动更顺畅，因为残差路径上没有 Norm 阻碍。缺点是理论上 Post-Norm 收敛后的效果上限可能略高，但在实践中 Pre-Norm 已成为大模型的标配。

---

## ✅ 自测题

### 题目 1（选择题）

在 Self-Attention 中，Q、K、V 分别代表什么角色?

A. Q 是输入，K 是权重，V 是偏置
B. Q 是查询，K 是键，V 是值；Q 和 K 计算匹配度，用匹配度加权 V
C. Q 是编码器输出，K 是解码器输入，V 是最终输出
D. Q、K、V 是三种不同的激活函数

▼ **查看答案**

**B**。Q（Query）代表"我在找什么"，K（Key）代表"我有什么"，Q 和 K 的点积计算匹配度（注意力分数），经 softmax 归一化后作为权重，对 V（Value）做加权求和得到输出。

### 题目 2（选择题）

RNN 被 Transformer 取代的主要原因是什么？（多选）

A. RNN 的长距离依赖问题——信息在传递过程中逐渐丢失
B. RNN 无法并行计算，训练速度慢
C. RNN 的参数量太大
D. RNN 不能处理中文

▼ **查看答案**

**A 和 B**。RNN 的两大核心缺陷是：(1) 长距离依赖问题，序列越长信息丢失越严重；(2) 串行计算导致无法利用 GPU 并行加速。C 错误（RNN 参数量通常比 Transformer 小）；D 错误（RNN 可以处理任何语言）。

### 题目 3（简答题）

请画出或描述 MiniMind 一个 Transformer Block 的完整结构（包括 Norm、Attention、FFN、残差连接的顺序）。

▼ **查看答案**

MiniMind 使用 Pre-Norm 结构，一个 Block 的流程是：

```
输入 x
  → RMSNorm(x)
  → Self-Attention（GQA，带 RoPE 和因果 mask）
  → + x（残差连接）→ 得到 x'
  → RMSNorm(x')
  → FFN（SwiGLU）
  → + x'（残差连接）→ 输出
```

关键点：(1) 归一化在子层之前（Pre-Norm）；(2) 每个子层后都有残差连接；(3) Attention 使用 GQA（8 个 Q 头，4 个 KV 头）和 RoPE；(4) FFN 使用 SwiGLU 激活。

---

## 🔮 下一节预告

Transformer 的原理搞明白了，但要真正看懂 MiniMind 的代码，我们还需要一个**工具**——PyTorch。下一节我们将快速上手 PyTorch，学会 Tensor 操作、自动求导、以及如何用代码搭建一个简单的神经网络。

**👉 L03 - PyTorch 快速上手：工欲善其事，必先利其器**